COSC 4368 Maze Solver Project
Second Check-In Report

Your Team Name

March 23, 2026

Objective

The goal of the second check-in was to:

1. load the maze in Python,
2. solve the maze without hazards using a naive method,
3. visualize the solution,
4. load hazards into the maze,
5. demonstrate that the hazards function correctly.

Maze Representation

The maze was represented internally as a 64×64 grid, matching the project specification. Logical cells were navigable positions, while walls were represented as blocked movement edges between neighboring cells. This preserves the original maze image structure while still supporting a 64×64 environment.

Naive Solve

Breadth-First Search (BFS) was used to solve the maze without hazards. BFS was selected because it is simple, deterministic, and guarantees a shortest path in an unweighted grid.

Visualization

The maze and BFS solution path were visualized using `matplotlib`. The solution path was highlighted in red for contrast.

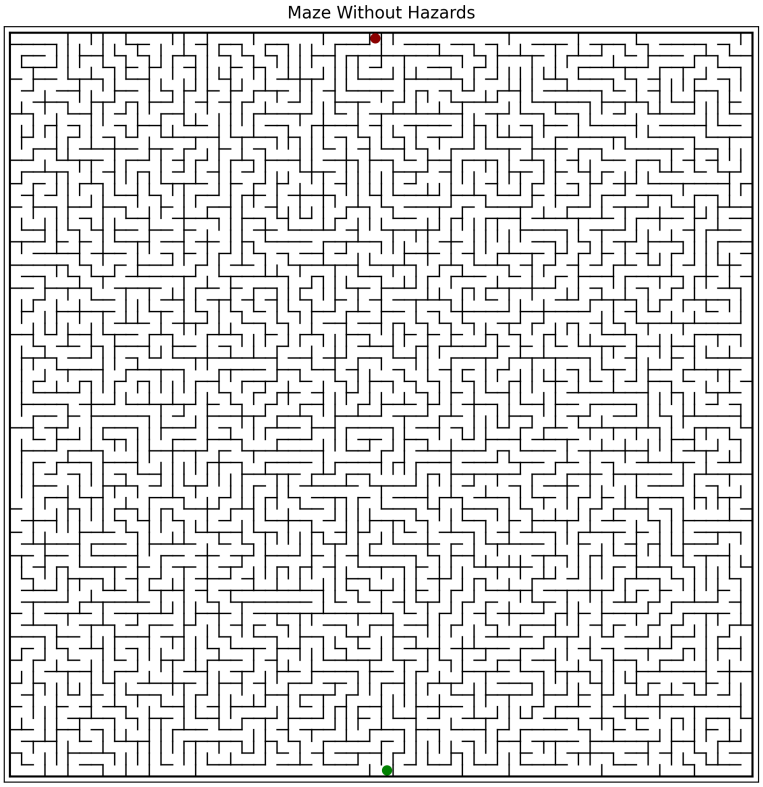

Figure 1: Maze loaded without hazards.

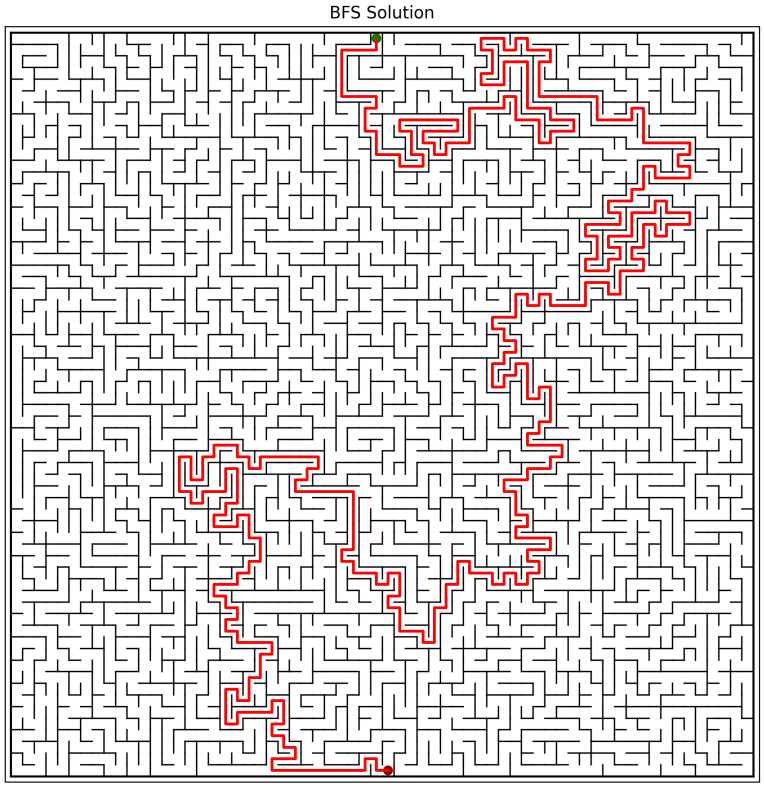

Figure 2: BFS solution path visualized on the maze.

Hazard Implementation

The environment supports:

- **Death pits:** immediate death, respawn at the start next turn,
- **Teleport pads:** deterministic teleport to a fixed destination,
- **Confusion pads:** inverted controls for the rest of the turn and the following turn.

Hazards were generated automatically at a density of 7% of navigable cells, which satisfies the required 5–10% range. A safe path from start to goal was preserved.

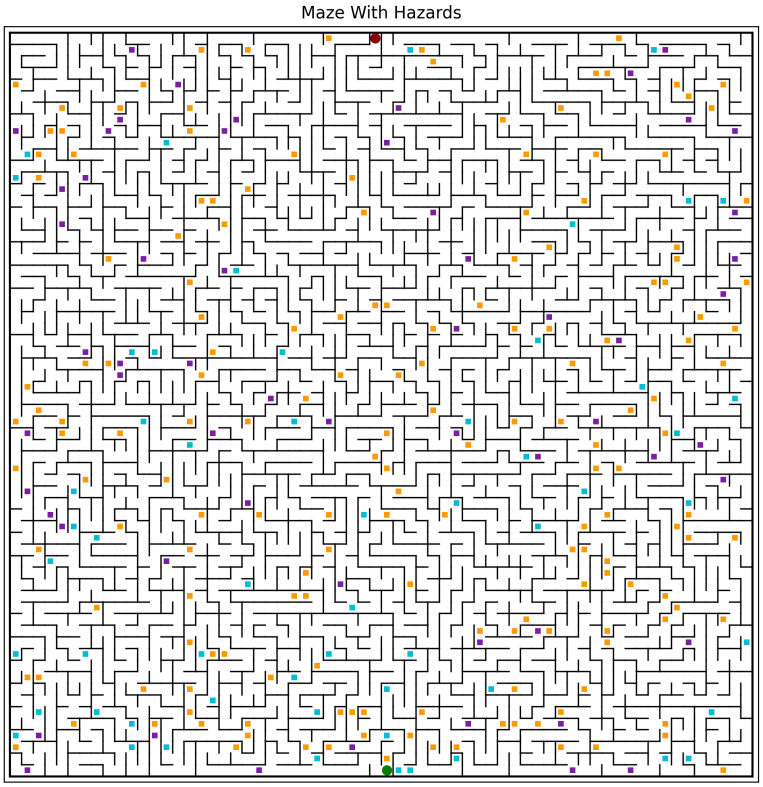

Figure 3: Maze with hazards loaded and displayed.

Hazard Demonstration

The hazard system was demonstrated with three scripted examples:

- stepping on a death pit,
- stepping on a teleport pad,
- stepping on a confusion pad and showing inverted controls.

Environment API

The environment matches the project turn format:

- each turn accepts between 1 and 5 actions,
- actions execute sequentially,
- remaining actions are ignored if the agent dies or reaches the goal,
- the environment returns a `TurnResult` object with wall hits, current position, death status, confusion status, goal status, teleport status, and actions executed.

AI Usage Disclosure

AI assistance was used during development.

Tool used: ChatGPT

Prompts used:

- "Help me finish my second check in. I've already coded out the maze construction from the picture. I need to do the rest."
- "Give me the full block code for `maze.py` and let me know if I need to update `maze_config.py`."
- "Let's finish all the other files now with the corrected `maze.py` code and have a working demo check in."

How AI performed: The AI helped with scaffolding for the environment, solver, hazard generation, visualization, and report drafting. All code was reviewed, edited, and integrated by the team.

Conclusion

The second check-in requirements were completed by loading the maze in Python, solving the no-hazard maze with BFS, visualizing the solution, loading hazards, and demonstrating the hazard behaviors.